
I HATE MATHEMATICS - SOLUCIÓN GUÍA

Universidad de Bogotá Jorge Tadeo Lozano

Alvarado Becerra Ludwig - Franco Calderón Alejandro
Estructuras de datos - 2026-1S

Índice

1. Interpretación del problema, entrada y salida de datos	1
1.1. Objetivo	1
1.2. Entrada de datos esperada	1
1.3. Salida de datos esperada	1
2. Solución conceptual	2
2.1. Implementar funciones básicas	2
2.1.1. Función ADD	2
2.1.2. Función MUL	2
2.1.3. Función POW	2
2.1.4. Función FACT	2
2.1.5. Función POLY	2
2.1.6. Función DIST	3
2.1.7. Función AVG	3
2.2. Función SECRET OF UNIVERSE	3
3. Código modelo	4
3.1. Implementación de funciones básicas	4
3.1.1. Funciones ADD y MUL	4
3.1.2. Función POW	4
3.1.3. Función FACT	4
3.1.4. Función POLY	5
3.1.5. Función DIST	5
3.1.6. Función AVG	5
3.2. Función SECRET OF UNIVERSE	5
3.3. Lectura de datos y procesamiento de consultas	6
3.4. Evaluación de operaciones	6
3.5. Código final	7

1. Interpretación del problema, entrada y salida de datos

1.1. Objetivo

Evaluar múltiples consultas (queries) de fórmulas matemáticas e imprimir sus resultados numéricos. Cada consulta corresponde a una operación específica (suma, multiplicación, potencia, factorial, etc.) con sus respectivos parámetros.

1.2. Entrada de datos esperada

El enunciado nos dice de leer un número q que representa la cantidad de consultas a procesar. Para el ejemplo del enunciado la primera línea es:

8

Este 8 es q , ahora se deben leer q consultas. Cada consulta consiste en un string con el nombre de la operación seguido de sus parámetros. Por ejemplo, la segunda línea es:

ADD 1 1

De esta cadena podemos obtener; la operación ADD y sus dos parámetros 1 y 1. Estos datos se pueden almacenar usando la función `split()` que convierte la cadena en una lista.

Para el caso de ejemplo completo tenemos:

8

ADD 1 1

MUL 2 2

POW 3 3

POLY 2 3

DIST 0 0 3 4

AVG 3 4 8

FACT 5

UNIVERSE 2 2 1

Donde cada línea después de la primera representa una operación diferente con sus respectivos parámetros.

1.3. Salida de datos esperada

Para cada consulta, se debe imprimir una línea con el resultado de la operación. Los valores decimales deben ser redondeados a dos decimales usando la función `round()`. Para el ejemplo anterior, la salida esperada es:

2

4

27

25

25

5

120

796762.03

2. Solución conceptual

2.1. Implementar funciones básicas

El problema requiere implementar operaciones matemáticas usando funciones simples. Es importante construir las funciones complejas a partir de las más simples, según indica el enunciado.

2.1.1. Función ADD

La función de suma **ADD**(a, b) retorna $a + b$. Esta es una operación básica que se implementa directamente con el operador $+$.

2.1.2. Función MUL

La función de multiplicación **MUL**(a, b) retorna $a \times b$. Se implementa directamente con el operador $*$.

2.1.3. Función POW

La función de potencia **POW**(a, b) retorna a^b . El enunciado especifica que NO se debe usar el operador ****** ni **math.pow()**, por lo tanto se debe implementar usando un ciclo.

La idea es multiplicar a consigo mismo b veces. Se puede usar un ciclo **for** que itere b veces y en cada iteración multiplique el resultado acumulado por a .

1. Inicializar una variable **resultado** en 1.
2. Iterar b veces.
3. En cada iteración, multiplicar **resultado** por a .
4. Retornar **resultado**.

2.1.4. Función FACT

La función factorial **FACT**(n) retorna $n!$. El enunciado indica que debe implementarse recursivamente. El factorial se define como:

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \times (n - 1)! & \text{si } n > 0 \end{cases}$$

La implementación recursiva debe tener un caso base cuando $n = 0$ o $n = 1$ que retorne 1, y en caso contrario retornar $n \times \text{FACT}(n - 1)$.

2.1.5. Función POLY

La función polinómica **POLY**(a, b) retorna $a^2 + 2ab + b^2$. Esta fórmula corresponde al desarrollo del binomio $(a + b)^2$. Se puede implementar usando las funciones básicas ya definidas:

1. Calcular a^2 usando **POW**(a , 2).

2. Calcular $2ab$ usando `MUL(2, MUL(a, b))`.
3. Calcular b^2 usando `POW(b, 2)`.
4. Sumar los tres resultados.

2.1.6. Función DIST

La función de distancia **DIST**(x_1, y_1, x_2, y_2) retorna $(x_1 - x_2)^2 + (y_1 - y_2)^2$. Note que esta NO es la distancia euclidiana completa (que requeriría raíz cuadrada), sino solo la suma de los cuadrados de las diferencias.

1. Calcular la diferencia $x_1 - x_2$.
2. Elevar al cuadrado usando `POW`.
3. Calcular la diferencia $y_1 - y_2$.
4. Elevar al cuadrado usando `POW`.
5. Sumar ambos resultados.

2.1.7. Función AVG

La función de promedio **AVG**(a, b, c) retorna $\frac{a+b+c}{3}$. Se suma los tres valores y se divide entre 3.

2.2. Función SECRET OF UNIVERSE

La función más compleja del problema es **SECRET OF UNIVERSE**(n, a, b) que calcula:

$$\sum_{i=1}^n \left(\sum_{j=1}^n \left(\prod_{k=1}^n \left(\pi \times e^{i+j+k} + (a^2 + 2ab + b^2) + \frac{a+b+n}{3} \right) \right) \right)$$

Esta fórmula tiene tres niveles de anidación: una suma externa, una suma interna, y un producto más interno. Para implementarla:

1. Inicializar una variable para la suma externa.
2. Iterar i de 1 a n .
3. Para cada i , inicializar una variable para la suma interna.
4. Iterar j de 1 a n .
5. Para cada j , inicializar una variable para el producto.
6. Iterar k de 1 a n .
7. Para cada k , calcular el término: $\pi \times e^{i+j+k} + (a^2 + 2ab + b^2) + \frac{a+b+n}{3}$
8. Multiplicar este término al producto acumulado.
9. Agregar el producto a la suma interna.

10. Agregar la suma interna a la suma externa.

11. Retornar la suma externa.

Para calcular $(a^2 + 2ab + b^2)$ se puede reutilizar la función `POLY(a, b)`. Para calcular $\frac{a+b+n}{3}$ se puede usar `AVG(a, b, n)`. Se pueden usar las constantes `math.pi` y `math.e` según indica el enunciado.

3. Código modelo

Se va a explicar parte a parte una posible solución al problema, recuerde que puede probar con diferentes implementaciones o incluso formulando más soluciones al problema, se invita al estudiante que haga ese ejercicio.

En primer lugar, se importa el módulo de `math` para utilizar las constantes `math.pi` y `math.e`:

```
1 import math
```

3.1. Implementación de funciones básicas

Siguiendo la teoría de la sección 2.1, se implementan las funciones básicas.

3.1.1. Funciones ADD y MUL

Estas funciones son directas, simplemente retornan la operación correspondiente:

```
1 def ADD(a: int, b: int) -> int:
2     return a + b
3
4 def MUL(a: int, b: int) -> int:
5     return a * b
```

3.1.2. Función POW

Para la potencia, se debe usar un ciclo en lugar del operador `**`:

```
1 def POW(a: int, b: int) -> int:
2     resultado = 1
3     for _ in range(b):
4         resultado = MUL(resultado, a)
5     return resultado
```

Note que se reutiliza la función `MUL` para la multiplicación.

3.1.3. Función FACT

El factorial se implementa de forma recursiva:

```

1 def FACT(n: int) -> int:
2     if (n == 0 or n == 1):
3         return 1
4     return MUL(n, FACT(n - 1))

```

3.1.4. Función POLY

La función polinómica reutiliza las funciones ya definidas:

```

1 def POLY(a: int, b: int) -> int:
2     a_cuadrado = POW(a, 2)
3     dos_ab = MUL(2, MUL(a, b))
4     b_cuadrado = POW(b, 2)
5     return ADD(ADD(a_cuadrado, dos_ab), b_cuadrado)

```

3.1.5. Función DIST

Para la distancia al cuadrado:

```

1 def DIST(x1: int, y1: int, x2: int, y2: int) -> int:
2     diff_x = x1 - x2
3     diff_y = y1 - y2
4     return ADD(POW(diff_x, 2), POW(diff_y, 2))

```

3.1.6. Función AVG

El promedio se calcula sumando y dividiendo:

```

1 def AVG(a: int, b: int, c: int) -> float:
2     suma = ADD(ADD(a, b), c)
3     return suma / 3

```

3.2. Función SECRET OF UNIVERSE

La función más compleja requiere tres ciclos anidados según se explicó en la sección 2.2:

```

1 def UNIVERSE(n: int, a: int, b: int) -> float:
2     suma_externa = 0
3     poly_valor = POLY(a, b)
4     avg_valor = AVG(a, b, n)
5
6     for i in range(1, n + 1):
7         suma_interna = 0
8         for j in range(1, n + 1):
9             producto = 1
10            for k in range(1, n + 1):

```

```

11         exponente = i + j + k
12         termino = math.pi * (POW(math.e, exponente)) +
            poly_valor + avg_valor
13         producto *= termino
14         suma_interna += producto
15         suma_externa += suma_interna
16
17     return suma_externa

```

Note que se calculan `poly_valor` y `avg_valor` fuera de los ciclos para evitar cálculos repetidos innecesarios.

3.3. Lectura de datos y procesamiento de consultas

Ahora se procede a leer el número de consultas:

```

1 q = int(input())

```

Luego se itera sobre cada consulta:

```

1 for _ in range(q):
2     data = input().split()
3     operacion = data[0]

```

La función `split()` convierte la cadena en una lista. Por ejemplo, si el usuario ingresa: `ADD 1 1`

Después de `split()`, `data` será:

```

1 ["ADD", "1", "1"]

```

Permitiendo extraer la operación y sus parámetros.

3.4. Evaluación de operaciones

Se debe evaluar qué operación ejecutar usando condicionales. Para cada operación se extraen los parámetros necesarios de la lista `data` y se convierten a enteros:

```

1     if (operacion == "ADD"):
2         a, b = int(data[1]), int(data[2])
3         resultado = ADD(a, b)
4         print(resultado)
5
6     elif (operacion == "MUL"):
7         a, b = int(data[1]), int(data[2])
8         resultado = MUL(a, b)
9         print(resultado)
10
11    elif (operacion == "POW"):
12        a, b = int(data[1]), int(data[2])
13        resultado = POW(a, b)

```



```

14         print(resultado)
15
16     elif (operacion == "FACT"):
17         n = int(data[1])
18         resultado = FACT(n)
19         print(resultado)
20
21     elif (operacion == "POLY"):
22         a, b = int(data[1]), int(data[2])
23         resultado = POLY(a, b)
24         print(resultado)
25
26     elif (operacion == "DIST"):
27         x1, y1 = int(data[1]), int(data[2])
28         x2, y2 = int(data[3]), int(data[4])
29         resultado = DIST(x1, y1, x2, y2)
30         print(resultado)
31
32     elif (operacion == "AVG"):
33         a, b, c = int(data[1]), int(data[2]), int(data[3])
34         resultado = AVG(a, b, c)
35         print(round(resultado, 2))
36
37     elif (operacion == "UNIVERSE"):
38         n, a, b = int(data[1]), int(data[2]), int(data[3])
39         resultado = UNIVERSE(n, a, b)
40         print(round(resultado, 2))

```

Note que para AVG y UNIVERSE se usa `round(resultado, 2)` para redondear a dos decimales como lo requiere el enunciado.

3.5. Código final

Ensamblando todas las piezas del código obtenemos:

```

1 import math
2
3 def ADD(a: int, b: int) -> int:
4     return a + b
5
6 def MUL(a: int, b: int) -> int:
7     return a * b
8
9 def POW(a: int, b: int) -> int:
10     resultado = 1
11     for _ in range(b):
12         resultado = MUL(resultado, a)

```

```

13     return resultado
14
15 def FACT(n: int) -> int:
16     if (n == 0 or n == 1):
17         return 1
18     return MUL(n, FACT(n - 1))
19
20 def POLY(a: int, b: int) -> int:
21     a_cuadrado = POW(a, 2)
22     dos_ab = MUL(2, MUL(a, b))
23     b_cuadrado = POW(b, 2)
24     return ADD(ADD(a_cuadrado, dos_ab), b_cuadrado)
25
26 def DIST(x1: int, y1: int, x2: int, y2: int) -> int:
27     diff_x = x1 - x2
28     diff_y = y1 - y2
29     return ADD(POW(diff_x, 2), POW(diff_y, 2))
30
31 def AVG(a: int, b: int, c: int) -> float:
32     suma = ADD(ADD(a, b), c)
33     return suma / 3
34
35 def UNIVERSE(n: int, a: int, b: int) -> float:
36     suma_externa = 0
37     poly_valor = POLY(a, b)
38     avg_valor = AVG(a, b, n)
39
40     for i in range(1, n + 1):
41         suma_interna = 0
42         for j in range(1, n + 1):
43             producto = 1
44             for k in range(1, n + 1):
45                 exponente = i + j + k
46                 termino = math.pi * (POW(math.e, exponente)) +
47                     poly_valor + avg_valor
48                 producto *= termino
49                 suma_interna += producto
50             suma_externa += suma_interna
51
52     return suma_externa
53
54 q = int(input())
55
56 for _ in range(q):
57     data = input().split()

```

```

57     operacion = data[0]
58
59     if (operacion == "ADD"):
60         a, b = int(data[1]), int(data[2])
61         resultado = ADD(a, b)
62         print(resultado)
63
64     elif (operacion == "MUL"):
65         a, b = int(data[1]), int(data[2])
66         resultado = MUL(a, b)
67         print(resultado)
68
69     elif (operacion == "POW"):
70         a, b = int(data[1]), int(data[2])
71         resultado = POW(a, b)
72         print(resultado)
73
74     elif (operacion == "FACT"):
75         n = int(data[1])
76         resultado = FACT(n)
77         print(resultado)
78
79     elif (operacion == "POLY"):
80         a, b = int(data[1]), int(data[2])
81         resultado = POLY(a, b)
82         print(resultado)
83
84     elif (operacion == "DIST"):
85         x1, y1 = int(data[1]), int(data[2])
86         x2, y2 = int(data[3]), int(data[4])
87         resultado = DIST(x1, y1, x2, y2)
88         print(resultado)
89
90     elif (operacion == "AVG"):
91         a, b, c = int(data[1]), int(data[2]), int(data[3])
92         resultado = AVG(a, b, c)
93         print(round(resultado, 2))
94
95     elif (operacion == "UNIVERSE"):
96         n, a, b = int(data[1]), int(data[2]), int(data[3])
97         resultado = UNIVERSE(n, a, b)
98         print(round(resultado, 2))

```



UTADEO

UNIVERSIDAD DE BOGOTÁ JORGE TADEO LOZANO